LAPORAN PRAKTIKUM

STRUKTUR DATA

STACK

disusun Oleh:

RIFKI MAULANA

NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM : ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 19 APRIL 2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul "**STACK**". Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data STACK

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 19 April 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM 1
KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN 2
2.1 Dasar Teori 2
2.2 Kode Program 2
2.2.1 NilaiMaksimum 2
2.2.2 SiswaStack 3
2.2.3 stackArray 4
2.2.4 stackArrayDriver 5
2.2.5 StackPostfix 7
BAB III KESIMPULAN DAN SARAN 8
3.1 Kesimpulan 8
DAFTAR PUSTAKA 9

DAFTAR GAMBAR

Gambar Output 2. 1 3
Gambar Output 2. 2 4
Gambar Output 2. 3 6
Gambar Output 2. 4 7

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data adalah cara untuk menyimpan dan mengatur data sehingga Anda dapat menggunakannya secara efisien. Dalam Java, Stack adalah struktur data linier yang mengikuti prinsip *Last In First Out* (LIFO) dan didefinisikan dalam paket *java.util.* Secara internal, ia memperluas kelas *Vector*.

Stack mempertahankan urutan penyisipan dan mengizinkan nilai duplikat dan nilai null. Stack berkembang secara dinamis ketika kapasitasnya terlampaui. Semua metode Stack disinkronkan. Kelas Stack mengimplementasikan antarmuka *List, RandomAccess, Cloneable, dan Serializable.*

1.2 Tujuan Praktikum

- Memahami konsep *Abstract Data Type* (ADT) *Stack* dalam pemrograman.
- Mampu merancang dan mengimplementasikan *Stack* menggunakan bahasa Java.
- Menerapkan konsep enkapsulasi melalui atribut, constructor, accessor, dan mutator.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penerapan *Stack* dalam kasus kehidupan nyata.
- Melatih kemampuan membuat program yang terstruktur.
- Membantu memahami penggunaan class sebagai representasi objek di Java.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Kelas Stack merepresentasikan tumpukan objek *last-in-first-out* (LIFO). Kelas ini memperluas kelas Vector dengan lima operasi yang memungkinkan vektor diperlakukan sebagai tumpukan. Operasi push dan pop yang biasa disediakan, serta metode untuk mengintip item teratas pada tumpukan, metode untuk menguji apakah tumpukan kosong , dan metode untuk mencari item dalam tumpukan dan mengetahui seberapa jauh item tersebut dari puncak..

Method method dasar yang ada pada Stack antara lain:

1. *isEmpty*()

   cek apakah stack kosong atau tidak.

2. *peek*()

   untuk melihat objek teratas.

3. *pop*()

   menghapus sambil memberi return objek teratas.

4. *push*()

   memasukkan objek pada stack.

2.2 Kode Program

2.2.1 NilaiMaksimum

```
package pekan3_2511533007;
import java.util.Stack;

public class NilaiMaksimum_2511533007 {
        public static int max_3007(Stack<Integer> s_3007) {
                Stack<Integer> backup_3007 = new Stack<Integer>();
                int maxValue_3007 = s_3007.pop();
                backup_3007.push(maxValue_3007);
                while (!s_3007.isEmpty()) {
                        int next_3007 = s_3007.pop();
                        backup_3007.push(next_3007);
                        maxValue_3007 = Math.max(maxValue_3007, next_3007);
                }
                while (!backup_3007.isEmpty()) {
                        s_3007.push(backup_3007.pop());
                }
                return maxValue_3007;

        }
        public static void main(String[] args) {
                Stack<Integer> s_3007 = new Stack<Integer>();
                s_3007.push(70);
                s_3007.push(12);
                s_3007.push(14);
                System.out.println("isi stack ");
                System.out.println("Stack Teratas "+ s_3007.peek());
                System.out.println("Nilai maksimum " +max_3007(s_3007));
```

```
        }

}
```

Kode Program 2.1

Kode ini bertujuan mencari nilai maksimum dalam sebuah Stack. Pada method max_3007(), semua elemen di-pop dari stack asli ke stack backup sambil membandingkan nilai terbesar menggunakan Math.max(). Setelah selesai, semua elemen dikembalikan lagi ke stack asli agar isinya tidak berubah. Di main(), stack diisi dengan nilai 70, 12, dan 14, lalu program mencetak elemen teratas (14) dan nilai maksimum (70).

```
isi stack
Stack Teratas 14
Nilai maksimum 70
```

Gambar Output 2. 1

2.2.2 SiswaStack

```
package pekan3_2511533007;
import java.util.ArrayList;

class Siswa_3007 {
        String nama_3007;
        int nim_3007;

        public Siswa_3007(String nama_3007, int nim_3007) {
                this.nama_3007 = nama_3007;
                this.nim_3007 = nim_3007;
        }

        @Override
        public String toString() {
                return "Nim: " + nim_3007 + ", Nama: " + nama_3007;
        }
}

public class SiswaStack_2511533007 {
        private ArrayList<Siswa_3007> stack_3007;

        public SiswaStack_2511533007() {
                stack_3007 = new ArrayList<>();

        }

        public void push_3007(Siswa_3007 mhs_3007) {
                stack_3007.add(mhs_3007);
        }
        public Siswa_3007 pop_3007() {
                if (!isEmpty_3007()) {
                        return stack_3007.remove(stack_3007.size() - 1);
                }
                return null;
        }
        public Siswa_3007 peek_3007() {
                if (!isEmpty_3007()) {
                        return stack_3007.get(stack_3007.size() - 1);
                }
                return null;
        }

        public boolean isEmpty_3007() {
                return stack_3007.isEmpty();
        }
        public void tampilkanSiswa_3007() {
```

```
                for (int i = stack_3007.size() - 1; i >= 0; i--) {
                        System.out.println(stack_3007.get(i));
                }
        }
        public static void main(String[] args) {
                SiswaStack_2511533007 studentStack_3007 = new SiswaStack_2511533007();

                Siswa_3007 mhs1_3007 = new Siswa_3007("Alis", 1);
                Siswa_3007 mhs2_3007 = new Siswa_3007("Boby", 2);
                Siswa_3007 mhs3_3007 = new Siswa_3007("Charles", 3);

                studentStack_3007.push_3007(mhs1_3007);
                studentStack_3007.push_3007(mhs2_3007);
                studentStack_3007.push_3007(mhs3_3007);

                System.out.println("Siswa di dalam stack:");
                studentStack_3007.tampilkanSiswa_3007();

                System.out.println("siswa teratas " + studentStack_3007.peek_3007());
                System.out.println("mengeluarkan siswa teratas dari stack: " +
studentStack_3007.pop_3007());
                System.out.println("daftar siswa setelah di pop :");
                studentStack_3007.tampilkanSiswa_3007();

        }

}
```

Kode Program 2.2

Kode ini mengimplementasikan Stack berbasis ArrayList untuk menyimpan data siswa. Class Siswa_3007 menyimpan atribut nama dan nim. Class SiswaStack_2511533007 memiliki operasi stack yaitu push (menambah), pop (menghapus teratas), peek (melihat teratas), dan tampilkanSiswa (menampilkan semua isi stack dari atas ke bawah).

Di main(), tiga siswa (Alis, Boby, Charles) dimasukkan ke stack. Program lalu menampilkan isi stack, mencetak siswa teratas (Charles), melakukan pop untuk mengeluarkan Charles, kemudian menampilkan sisa stack yang tersisa (Boby dan Alis).

```
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Alis
siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
daftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Alis
```

Gambar Output 2. 2

2.2.3 stackArray

```
package pekan3_2511533007;

public class stackArray_2511533007 {
        static final int MAX_3007 = 1000;
        int top_3007;
        int a_3007[] =new int [MAX_3007];
        boolean isEmpty_3007() {
                return (top_3007 < 0);
```

```java
        }
        stackArray_2511533007() {
                top_3007 = -1;
        }
        boolean push_3007 (int x) {
                if (top_3007 >= (MAX_3007 - 1)) {
                        System.out.println("Stack Overflow");
                        return false;
                }else {
                        a_3007[++top_3007] = x;
                        System.out.println(x + " dimasukkan dalam stack");
                        return true;
                }
        }
        int pop_3007() {
                if (top_3007 < 0) {
                        System.out.println("Stack Underflow");
                        return 0;
                }
                else {
                        int x = a_3007[top_3007--];
                        return x;
                }
        }
        int peek_3007() {
                if (top_3007 < 0) {
                        System.out.println("Stack Underflow");
                        return 0;
                }
                else {
                        int x = a_3007[top_3007];
                        return x;
                }
        }
        void print_3007() {
                for (int i = top_3007; i > -1; i--) {
                        System.out.print(" " + a_3007[i]);
                }
        }

}
```

Kode Program 2.3

Kode ini mengimplementasikan Stack menggunakan Array dengan kapasitas maksimal 1000 elemen. Stack diinisialisasi dengan top = -1 sebagai tanda stack kosong. Terdapat 4 operasi utama yaitu push (menambah elemen, cek *overflow* jika penuh), pop (menghapus elemen teratas, cek *underflow* jika kosong), peek (melihat elemen teratas tanpa menghapus), dan print (menampilkan semua isi stack dari atas ke bawah). Penanda top_3007 selalu bergerak naik saat push dan turun saat pop. Program masih belum bisa dijalankan.

2.2.4 stackArrayDriver

```java
package pekan3_2511533007;

public class stackArrayDriver_2511533007 {
        public static void main (String[] args) {
                stackArray_2511533007 s = new stackArray_2511533007();
                s.push_3007(10);
                s.push_3007(20);
                s.push_3007(30);
                System.out.println(s.pop_3007() + " dikeluarkan dari stack");
                System.out.println("Elemen teratas adalah :" + s.peek_3007());
                System.out.print("Element pada stack :");
```

```
                s.print_3007();
        }

}
```

Kode Program 2.4

Kode ini adalah class driver untuk menguji stackArray. Program memasukkan tiga nilai (10, 20, 30) ke stack, lalu melakukan pop yang mengeluarkan 30 (elemen teratas), kemudian peek menampilkan 20 sebagai elemen teratas baru, dan terakhir mencetak sisa isi stack yaitu 20 dan 10.

```
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Element pada stack : 20 10
```

Gambar Output 2. 3

2.2.5 StackPostfix

```java
package pekan3_2511533007;
import java.util.Scanner;

public class StackPostfix_2511533007 {
    public static int postfixEvaluate_3007(String expression) {
        Stack<Integer> s_3007 = new Stack<Integer>();
        Scanner input_3007 = new Scanner(expression);
        while (input_3007.hasNext()) {
            if (input_3007.hasNextInt()) {      //an operand (integer)
                s_3007.push(input_3007.nextInt());
            } else {                            // an operator
                String operator_3007 = input_3007.next();
                int operand2_3007 = s_3007.pop();
                int operand1_3007 = s_3007.pop();
                if (operator_3007.equals("+")) {
                    s_3007.push(operand1_3007 + operand2_3007);
                } else if (operator_3007.equals("-")) {
                    s_3007.push(operand1_3007 - operand2_3007);
                } else if (operator_3007.equals("*")) {
                    s_3007.push(operand1_3007 * operand2_3007);
                } else {
                    s_3007.push(operand1_3007 / operand2_3007);
                }
            }
        }
        input_3007.close();
        return s_3007.pop();
    }
    public static void main(String[] args) {
        System.out.println("hasil postfix= " + postfixEvaluate_3007("5 2 4 * 7 -"));
    }

}
```

Kode Program 2.5

Kode ini mengevaluasi ekspresi postfix "5 2 4 * 7 -" menggunakan Stack. Program membaca input satu per satu baik itu operator maupun angka, jika input berupa angka maka di-push ke stack, jika berupa operator maka pop dua elemen teratas lalu dihitung dan hasilnya di-push kembali. Prosesnya yaitu 2 * 4 = 8, lalu 8 - 7 = 1 dan angka 5 tidak terlibat operasi apapun. Hasil akhir yang dikembalikan adalah elemen teratas stack yaitu 1

```
hasil postfix= 1
```

Gambar Output 2. 4

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Praktikum ini membahas implementasi struktur data Stack dalam Java melalui beberapa pendekatan dan studi kasus. Stack adalah struktur data yang bekerja dengan prinsip LIFO (*Last In, First Out*), di mana elemen yang terakhir dimasukkan adalah yang pertama dikeluarkan.

Dari praktikum ini, Stack dapat diimplementasikan dengan tiga cara yaitu menggunakan Array (stackArray), menggunakan ArrayList (SiswaStack), dan menggunakan class Stack bawaan Java (java.util.Stack). Setiap implementasi memiliki method dasar yang sama yaitu push() untuk menambah elemen, pop() untuk menghapus elemen teratas, peek() untuk melihat elemen teratas, dan isEmpty() untuk mengecek apakah stack kosong.

Selain implementasi dasar, praktikum ini juga menerapkan Stack pada studi kasus nyata seperti pencarian nilai maksimum menggunakan stack backup agar isi stack tetap terjaga, penyimpanan data objek seperti data siswa yang membuktikan stack tidak hanya bisa menyimpan tipe primitif, serta evaluasi ekspresi postfix yang membuktikan bahwa Stack sangat berguna dalam pemrosesan ekspresi matematika.

DAFTAR PUSTAKA

[1] GeeksforGeeks, "Stack Class in Java," [Daring]. Tersedia pada: https://www.geeksforgeeks.org/java/stack-class-in-java/. [Diakses: 19-Apr-2026].

[2] Oracle, "Stack (Java Platform SE 8)," [Daring]. Tersedia pada: https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html. [Diakses: 19-Apr-2026].

Laporan Tugas

a. Kode program

```java
package pekan3_2511533007;

public class Website_2511533007 {

    private String judul_3007;
    private String URL_3007;

    //constructor
    public Website_2511533007(String judul_3007, String URL_3007) {
        this.judul_3007 = judul_3007;
        this.URL_3007 = URL_3007;
    }

    //selektor/getter
    public String getJudul_3007() {
        return judul_3007;
    }
    public String getURL_3007() {
        return URL_3007;
    }

    //mutator/setter
    public void setJudul_3007(String judul_3007) {
        this.judul_3007 = judul_3007;
    }
    public void setURL_3007(String URL_3007) {
        this.URL_3007 = URL_3007;
    }

    @Override
    public String toString() {
        return "Judul: " + judul_3007 + "URL: " + URL_3007;
    }

}
```

Kode Program ADT

```java
package pekan3_2511533007;

import java.util.Stack;

public class Browser_2511533007 {

    //menampilkan pilihan yang bisa dipilih user
    public static void tampilkanInput_3007() {
        System.out.println("=== Browser History ===");
        System.out.println("1. Kunjungi Website");
        System.out.println("2. Tombol Back");
        System.out.println("3. Lihat Halaman Aktif");
        System.out.println("4. Cek Status History");
        System.out.println("5. Keluar");
    }
    // memasukkan data history kedalam stack(push)
    public static void tambahHistory_3007(Stack<Website_2511533007> stack_3007, Scanner input_3007) {
        System.out.println("Masukkan Judul: ");
        String judul_3007 = input_3007.nextLine();
        System.out.println("Masukkan URL: ");
        String URL_3007 = input_3007.nextLine();
        stack_3007.push(new Website_2511533007(judul_3007, URL_3007));
        System.out.println("Berhasil mengunjungi halaman");
    }
    // menampilkan halaman saat ini(peek)
    public static void tampilkanHalaman_3007(Stack<Website_2511533007> stack_3007 ) {
        if (stack_3007.isEmpty()) {
            System.out.println("Browser history kosong.");
        } else {
            System.out.println("Halaman saat ini: \n" + stack_3007.peek());
        }
    }
    // menghapus history teratas(pop)
    public static void hapusHistory_3007(Stack<Website_2511533007> stack_3007, Scanner input_3007) {
        if (stack_3007.isEmpty()) {
            System.out.println("History kosong");
        } else {
            System.out.println("History yang dihapus: " + stack_3007.pop());
        }
    }
    //memberi tahu jumlah history user
    public static void tampilkanJumlahHistory_3007(Stack<Website_2511533007> stack_3007) {
        if (stack_3007.isEmpty()) {
            System.out.println("History kosong");
        } else {
            System.out.println("Total history: " + stack_3007.size());
        }
    }


    public static void main (String[] args) {
        Stack<Website_2511533007> stacking_3007 = new Stack<>();
        Scanner input_3007 = new Scanner(System.in);
        int choice_3007;
        do {
            tampilkanInput_3007();
```

```java
            System.out.print("Pilih menu: ");
            choice_3007 = input_3007.nextInt();
            input_3007.nextLine();

            switch (choice_3007) {
                case 1:
                    tambahHistory_3007(stacking_3007, input_3007);
                    break;
                case 2:
                    hapusHistory_3007(stacking_3007, input_3007);
                    break;
                case 3:
                    tampilkanHalaman_3007(stacking_3007);
                    break;
                case 4:
                    tampilkanJumlahHistory_3007(stacking_3007);
                    break;
                case 5:
                    System.out.println("Keluar dari program.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }
        } while (choice_3007 != 5);

        input_3007.close();
    }
}
```

Kode Program Implementasi ADT

b. Penjelasan kode program

Kedua program ini mensimulasikan fitur history pada browser menggunakan Stack. Class Website_2511533007 berfungsi sebagai ADT yang menyimpan atribut judul dan URL sebuah website beserta getter, setter, dan Overridenya.

Class Browser_2511533007 adalah program utama dengan 5 pilihan yaitu kunjungi website (push data website ke stack), tombol back (pop untuk menghapus history teratas), lihat halaman aktif (peek untuk melihat website saat ini), cek status history (menampilkan jumlah history), dan keluar. Program berjalan dalam loop do-while hingga user memilih keluar. Konsep ini mencerminkan cara kerja nyata tombol *back* pada browser yang juga menggunakan prinsip LIFO. Program berjalan dalam loop do-while hingga pengguna memilih menu keluar.